PLAYWRIGHT FINGERTIPS
THE COMPLETE QUICK REFERENCE

Every locator, action, assertion and combo - one page-flip away

Prepared for: jashu5446
29 September 2026

INDEX

PLAYWRIGHT FINGERTIPS — THE COMPLETE QUICK REFERENCE 3

SECTION 1 — LOCATORS (every way to FIND) 4

SECTION 2 — ACTIONS (every way to DO) 5

SECTION 3 — TEXT & VALUE READERS (the getText family — know the differences!) 6

SECTION 4 — FILE UPLOAD & DOWNLOAD (+ the FileChooser class) 7

SECTION 5 — FRAMES (+ the FrameLocator class) 8

SECTION 6 — TABS, WINDOWS & CONTEXTS (the Browser family) 9

SECTION 7 — DIALOGS (the Dialog class) 10

SECTION 8 — WAITING (every wait, and when to use each) 11

SECTION 9 — THE BUILT-IN CLASSES (know each by NAME + purpose) 12

SECTION 10 — KEYBOARD & MOUSE 13

SECTION 11 — NETWORK & API (mocking + API testing) 14

SECTION 12 — ASSERTIONS (every expect, by type) 15

SECTION 13 — CLI FLAGS (run-time controls) 17

SECTION 14 — POWER COMBOS (the daily recipes — memorize these five) 18

PLAYWRIGHT FINGERTIPS — THE COMPLETE QUICK REFERENCE

Every locator, action, built-in class, reader method, assertion and command — one line each, with an example and a pro-tip to memorize it.

SECTION 1 — LOCATORS (every way to FIND)

Method	Finds by	Example
`getByRole()`	ARIA role + accessible name — THE best choice	`page.getByRole('button', { name: 'Sign in' })`
`getByLabel()`	a form field's `<label>` text	`page.getByLabel('Email')`
`getByPlaceholder()`	the placeholder text	`page.getByPlaceholder('Search products')`
`getByText()`	visible text (substring by default)	`page.getByText('Welcome back')`
`getByAltText()`	an image's alt attribute	`page.getByAltText('Company logo')`
`getByTitle()`	the title attribute	`page.getByTitle('Submit order')`
`getByTestId()`	`data-testid` — the escape hatch	`page.getByTestId('cart-count')`
`locator()`	raw CSS	`page.locator('#submit')`
`locator()`	raw XPath (prefix `xpath=`)	`page.locator('xpath=//button[@type="submit"]')`
`getByRole()` + `exact: true`	exact full-name match (no substring)	`page.getByRole('button', { name: 'Pay', exact: true })`
regex name	match names by pattern	`page.getByRole('link', { name: /cart/i })`

Locator refinement (narrowing down):

Method	Does	Example
`.filter({ hasText })`	keeps matches containing text	`page.getByRole('listitem').filter({ hasText: 'Laptop' })`
`.filter({ has })`	keeps matches CONTAINING a child locator	`page.getByRole('row').filter({ has: page.getByText('Laptop') })`
`.first()` / `.last()`	first / last match	`page.getByRole('row').first()`
`.nth(i)`	match number i (0-based)	`page.getByRole('row').nth(2)`
`.and()`	BOTH locators must match	`btn.and(page.getByRole('enabled'))`
`.or()`	EITHER locator matches	`page.getByLabel('Username').or(page.getByLabel('Email'))`
`.count()`	how many match (async!)	`const n = await page.getByRole('row').count();`
`.all()`	array of all matches	`for (const r of await page.getByRole('row').all())`
`.not()`	excludes a sub-locator	`page.getByRole('listitem').not(page.getByClass('ad'))`

MEMORY HOOK: *"Role first, Label for fields, TestId as escape, CSS/XPath as the last resort — and many matches means first/nth/filter, never guess."*

SECTION 2 — ACTIONS (every way to DO)

Method	Does	Example
`click()`	click (auto-waits for visible/stable/enabled)	`await page.getByRole('button', { name: 'Save' }).click();`
`click({ button })`	right / middle click	`.click({ button: 'right' })`
`click({ double })`	double-click (same as dblclick)	`.click({ double: true })`
`click({ force: true })`	skip actionability — DEBUG ONLY	`.click({ force: true })`
`click({ position })`	click at offset inside element	`.click({ position: { x: 10, y: 5 } })`
`dblclick()`	double-click	`await row.dblclick();`
`fill()`	REPLACE input value (clears first)	`await page.getByLabel('Email').fill('a@b.com');`
`clear()`	empty an input	`await page.getByLabel('Email').clear();`
`press()`	one key or combo while focused	`.press('Control+A')`
`pressSequentially()`	type char by char (like a human)	`.pressSequentially('selenium', { delay: 50 })`
`type()`	DEPRECATED — use pressSequentially	(avoid in interviews: say the new name)
`check()` / `uncheck()`	tick / untick (waits for checkable)	`await page.getByRole('checkbox').check();`
`setChecked(bool)`	set to desired state (no-op if already)	`.setChecked(true)`
`selectOption()`	choose in a `<select>`	`.selectOption('in')` by value
selectOption by label	choose by visible text	`.selectOption({ label: 'India' })`
selectOption by index	choose by position	`.selectOption({ index: 2 })`
selectOption multiple	multi-select	`.selectOption(['in', 'us'])`
`hover()`	mouse over (opens menus/tooltips)	`await page.getByRole('menuitem', { name: 'Billing' }).hover();`
`focus()` / `blur()`	focus / unfocus	`await box.focus();`
`dragTo()`	drag and drop	`await source.dragTo(target);`
`tap()`	touch tap (mobile emulation)	`await page.getByRole('button').tap();`
`scrollIntoViewIfNeeded()`	bring element into view	`await page.getByTestId('footer').scrollIntoViewIfNeeded();`
`highlight()`	flash the element on screen (debugging)	`await btn.highlight();`

MEMORY HOOK: *"fill replaces, press hits keys, check ticks, select picks, hover reveals, drag moves — and every one of them auto-waits."*

SECTION 3 — TEXT & VALUE READERS (the getText family — know the differences!)

Method	Returns	Sees hidden text?	Example
`textContent()`	raw text INCLUDING invisible text	YES	`const t = await page.getByRole('row').textContent();`
`innerText()`	text as the HUMAN sees it (visible only)	NO	`const t = await page.getByRole('row').innerText();`
`innerHTML()`	the HTML markup string	—	`const h = await page.getByRole('alert').innerHTML();`
`inputValue()`	what's typed in input/textarea/select	—	`const v = await page.getByLabel('Email').inputValue();`
`getAttribute()`	an attribute's value	—	`const href = await link.getAttribute('href');`
`allTextContents()`	ARRAY of textContent for every match	YES	`const all = await page.getByRole('row').allTextContents();`
`allInnerTexts()`	ARRAY of innerText for every match	NO	`const all = await page.getByRole('row').allInnerTexts();`
`toHaveText()` / `toContainText()`	ASSERT with retry (prefer over reading!)	—	`await expect(page.getByRole('h1')).toHaveText('Welcome');`

PRO TIP (the memorization rule):

> ***textContent = the WHOLE truth (even hidden) — innerText = what the EYES see — innerHTML = the markup — inputValue = what's TYPED — allTextContents = the array version — toHaveText = assert, don't read.*** *Golden rule: if your goal is to VERIFY, never read-then-compare (`if (text === 'x')`) — use `await expect(locator).toHaveText('x')` so the assertion retries and the failure message is useful.*

MEMORY HOOK: *"Verify with expect, read only when you must process."*

SECTION 4 — FILE UPLOAD & DOWNLOAD (+ the FileChooser class)

Method / Class	Does	Example
`setInputFiles()`	upload straight into `<input type="file">` — no dialog	`await page.setInputFiles('input[type="file"]', 'test-data/receipt.pdf');`
`setInputFiles()` multiple	upload several files	`.setInputFiles('input[type=file]', ['a.pdf', 'b.png'])`
`setInputFiles()` with buffer	upload from memory	`.setInputFiles(sel, { name: 'a.pdf', mimeType: 'application/pdf', buffer })`
FileChooser class	wraps the browser's file picker (no-input uploaders)	see below
`waitForEvent('filechooser')`	listen for the picker, then set	`const fc = page.waitForEvent('filechooser'); await dropzone.click(); (await fc).setFiles('a.pdf');`
`waitForEvent('download')`	listen for a download starting	`const [dl] = await Promise.all([page.waitForEvent('download'), link.click()]);`
Download class `.saveAs()`	save the downloaded file	`await dl.saveAs('downloads/report.csv');`
`Download.suggestedFilename()`	the name the server suggests	`const name = dl.suggestedFilename();`
`Download.path()`	temp path of the downloaded file	`const p = await dl.path();`

MEMORY HOOK: *"Input tag = setInputFiles. No input = FileChooser. Download = waitForEvent + saveAs."*

SECTION 5 — FRAMES (+ the FrameLocator class)

Method	Does	Example
`page.frameLocator()`	get a scoped "world" inside an iframe	`const editor = page.frameLocator('#editor');`
act inside the frame	locators chain on frameLocator	`await editor.getByLabel('Comment').fill('Hi');`
nested frames	chain frameLocators	`page.frameLocator('#outer').frameLocator('#inner')`
`page.frames()`	list ALL frames (incl. main)	`for (const f of page.frames()) console.log(f.url());`
`page.frame()`	get a Frame object by name/url	`const f = page.frame({ url: /editor/ });`

MEMORY HOOK: *"frameLocator IS the scope — no switching in, no defaultContent back out."*

SECTION 6 — TABS, WINDOWS & CONTEXTS (the Browser family)

Method	Does	Example
`browser.newContext()`	isolated incognito-like session	`const ctx = await browser.newContext();`
`context.newPage()`	open a tab inside a context	`const tab = await context.newPage();`
`page.waitForEvent('popup')`	capture a tab opened by a link	`const [pop] = await Promise.all([page.waitForEvent('popup'), link.click()]);`
`context.pages()`	ALL open tabs as an array	`const tabs = context.pages();`
find a tab by URL	recover a lost reference	`context.pages().find(p => p.url().includes('payment'))`
`page.bringToFront()`	make tab VISIBLE (cosmetic, not targeting)	`await tab.bringToFront();`
`page.opener()`	which tab opened this one	`const parent = await pop.opener();`
`page.close()`	close one tab	`await tab.close();`
`context.storageState()`	save cookies + localStorage	`await page.context().storageState({ path: '.auth/user.json' });`
`context.clearCookies()`	wipe cookies	`await context.clearCookies();`
`context.addCookies()`	inject cookies	`await context.addCookies([{ name: 'session', value: 'abc', url: 'https://app.com' }]);`

MEMORY HOOK: *"Browser is the building, Context the hotel room, Page the TV — every tab is a Page object; no switching exists."*

SECTION 7 — DIALOGS (the Dialog class)

Method	Does	Example
`page.on('dialog')`	handle alert/confirm/prompt BEFORE the trigger	`page.once('dialog', async d => { console.log(d.message()); await d.accept(); });`
`dialog.accept()`	click OK (optionally type into prompt)	`await d.accept('typed text');`
`dialog.dismiss()`	click Cancel	`await d.dismiss();`
`dialog.message()`	read the popup text	`const m = d.message();`
`dialog.defaultValue()`	a prompt's pre-filled value	`const v = d.defaultValue();`

MEMORY HOOK: *"Register the handler BEFORE the click — Playwright auto-dismisses otherwise."*

SECTION 8 — WAITING (every wait, and when to use each)

Method	Waits for	Example
(nothing)	actions auto-wait visible+stable+enabled+receives events	`await btn.click();`
`expect(locator).toBeVisible()`	assertion true (retries)	`await expect(page.getByRole('alert')).toBeVisible();`
`locator.waitFor()`	element to appear / detach	`await page.getByTestId('results').waitFor();`
`page.waitForURL()`	SPA URL change	`await page.waitForURL('**/dashboard');`
`page.waitForLoadState()`	load / domcontentloaded / networkidle	`await page.waitForLoadState('networkidle');`
`page.waitForResponse()`	a specific API response	`await page.waitForResponse(r => r.url().includes('/api/search') && r.status() === 200);`
`page.waitForRequest()`	a specific request going out	`await page.waitForRequest(r => r.url().includes('/api/login'));`
`page.waitForEvent()`	popup / download / console	`await page.waitForEvent('popup');`
`page.waitForTimeout()`	BLIND sleep — debug only, banned in suites	`await page.waitForTimeout(3000);`

MEMORY HOOK: *"Default to nothing (auto-wait), assert with expect, wait for URLs and APIs explicitly, and never sleep."*

SECTION 9 — THE BUILT-IN CLASSES (know each by NAME + purpose)

Class	One-line identity	Typical use
Playwright	the entry point / engine accessor	`playwright.chromium.launch()` (library mode)
BrowserType	one browser engine (chromium, firefox, webkit)	`chromium.launch()` returns a Browser
Browser	one running browser process (heavy)	`await browser.close();`
BrowserContext	isolated incognito session (cheap, fresh per test)	`await browser.newContext();`
Page	one TAB — the object you use 90% of the time	`await page.goto('/');`
Locator	a lazy FIND query, auto-waiting on every use	`page.getByRole('button')`
FrameLocator	the scoped world inside an iframe	`page.frameLocator('#f')`
APIRequestContext	the built-in API client (the `request` fixture)	`await request.get('/api/users');`
APIResponse	an API response object	`expect(res.status()).toBe(200);`
Route	an intercepted network call (mock/modify)	`await page.route('**/api/x', r => r.fulfill({...}))`
Dialog	a JS alert/confirm/prompt	`d.accept()`
Download	a started download	`dl.saveAs('f.csv')`
FileChooser	the OS file picker (no-input uploaders)	`fc.setFiles('a.pdf')`
ConsoleMessage	a console log line from the page	`page.on('console', m => ...)`
Keyboard	the keyboard of the page	`page.keyboard.press('Enter')`
Mouse	the mouse of the page	`page.mouse.wheel(0, 300)`
Touchscreen	touch input	`page.touchscreen.tap(10, 10)`
Worker	a web-worker process in the page	`page.on('worker', ...)`
TimeoutError	the error when a budget expires — READ it	error message says what it waited for
TestInfo / TestStep	run metadata / report steps	`test.info()`, `test.step()`

MEMORY HOOK: *"Page is your desk, Locator is the query, Context is the room — everything else is a specialist you call by name."*

SECTION 10 — KEYBOARD & MOUSE

Method	Does	Example
`keyboard.press()`	key or combo	`await page.keyboard.press('Control+A');`
`keyboard.down()/up()`	hold / release a key	`await page.keyboard.down('Shift');`
`keyboard.insertText()`	insert text (no key events)	`await page.keyboard.insertText('hello');`
`mouse.click()`	click at coordinates	`await page.mouse.click(100, 200);`
`mouse.dblclick()`	double-click at coordinates	`await page.mouse.dblclick(100, 200);`
`mouse.down()/up()`	press/release at current position	drag building blocks
`mouse.move()`	move to coordinates (with steps for smoothness)	`await page.mouse.move(100, 200, { steps: 10 });`
`mouse.wheel()`	scroll	`await page.mouse.wheel(0, 500);`

MEMORY HOOK: *"Keyboard presses, Mouse points — coordinates only when locators can't."*

SECTION 11 — NETWORK & API (mocking + API testing)

Method	Does	Example
`page.route()`	intercept matching requests	`await page.route('**/api/orders', handler);`
`route.fulfill()`	MOCK a response	`route.fulfill({ status: 500, body: '{"error":"down"}' })`
`route.abort()`	BLOCK a request	`route.abort()`
`route.fetch()`	call the REAL API, then modify	`const resp = await route.fetch();`
`page.unroute()`	stop intercepting	`await page.unroute('**/api/x');`
`request.get()`	API GET (fixture, no browser)	`const res = await request.get('/api/users/1');`
`request.post()`	API POST with body	`await request.post('/api/orders', { data: { item: 'book' } });`
`request.put/patch/delete()`	the rest of CRUD	`await request.delete('/api/orders/9');`
`res.status()` / `res.ok()`	status code / 2xx check	`expect(res.status()).toBe(200);`
`res.json()` / `res.text()`	parse the body	`const body = await res.json();`
`res.headers()`	response headers	`const ct = res.headers()['content-type'];`

MEMORY HOOK: *"fulfill fakes, fetch shapes, abort blocks, request tests — one WebSocket to see it all."*

SECTION 12 — ASSERTIONS (every expect, by type)

Positive state:

Assertion	Passes when	Example
`toBeVisible()`	element is rendered	`await expect(page.getByRole('h1')).toBeVisible();`
`toBeHidden()`	element not rendered	`await expect(page.getByTestId('spinner')).toBeHidden();`
`toBeEnabled()`	not disabled	`await expect(submitBtn).toBeEnabled();`
`toBeDisabled()`	disabled	`await expect(submitBtn).toBeDisabled();`
`toBeEditable()`	input accepts typing	`await expect(page.getByLabel('Email')).toBeEditable();`
`toBeEmpty()`	input has no value	`await expect(page.getByLabel('Email')).toBeEmpty();`
`toBeChecked()`	checkbox/radio ticked	`await expect(page.getByRole('checkbox')).toBeChecked();`
`toBeFocused()`	has keyboard focus	`await expect(page.getByLabel('Search')).toBeFocused();`

Content:

Assertion	Passes when	Example
`toHaveText()`	EXACT full text	`await expect(page.getByTestId('total')).toHaveText('Rs. 450');`
`toContainText()`	contains the text	`await expect(page.getByRole('alert')).toContainText('Order placed');`
`toHaveValue()`	input value equals	`await expect(page.getByLabel('Qty')).toHaveValue('3');`
`toHaveValues()`	multi-select values	`await expect(list).toHaveValues(['in', 'us']);`
`toHaveCount()`	number of matches	`await expect(page.getByRole('row')).toHaveCount(5);`

Attributes, identity, page:

Assertion	Passes when	Example
`toHaveAttribute()`	attribute value	`await expect(link).toHaveAttribute('href', '/docs');`
`toHaveClass()`	class attribute	`await expect(btn).toHaveClass(/active/);`
`toHaveCSS()`	a CSS property	`await expect(btn).toHaveCSS('color', 'rgb(0, 0, 255)');`
`toHaveId()`	the id	`await expect(el).toHaveId('submit');`
`toHaveAccessibleName()`	a11y name	`await expect(btn).toHaveAccessibleName('Sign in');`
`toHaveRole()`	a11y role	`await expect(page.getByTestId('x')).toHaveRole('button');`
`page` + `toHaveURL()`	page URL (regex/string)	`await expect(page).toHaveURL(/dashboard/);`
`page` + `toHaveTitle()`	tab title	`await expect(page).toHaveTitle(/Checkout/);`
`toHaveScreenshot()`	visual match vs baseline	`await expect(page).toHaveScreenshot('home.png');`

The special assertion TYPES:

Type	What it is	Example
negative `.not.`	inverts any assertion	`await expect(page.getByTestId('ad')).not.toBeVisible();`
`expect.soft()`	collect ALL failures (TestNG SoftAssert)	`await expect.soft(f1).toBeVisible();`

Type	What it is	Example
`expect.poll()`	retry a NON-locator value	`await expect.poll(async () => cartCount()).toBeGreaterThan(3);`
`toPass()`	retry an async block until it stops throwing	`await expect(async () => { ... }).toPass({ timeout: 10_000 });`
`.isVisible()`	boolean NOW — for `if`, NOT for asserting	`if (await banner.isVisible()) { ... }`
custom timeout per assertion	override the 5s default	`.toBeVisible({ timeout: 10_000 })`

PRO TIP (the assertion decision tree): *verifying state?* `expect(locator).toBe...`. *Reading to process?* `.textContent()`. *Branching?* `.isVisible()`. *Non-locator value that changes?* `expect.poll`. *Many checks in one test?* `expect.soft`.

SECTION 13 — CLI FLAGS (run-time controls)

Flag	Does	Example
`--headed`	show the browser	`npx playwright test --headed`
`--debug`	Inspector step-through	`npx playwright test --debug`
`--ui`	UI Mode with time travel	`npx playwright test --ui`
`--project=`	run one suite/browser	`npx playwright test --project=smoke`
`--grep`	filter by title/tag	`npx playwright test --grep "@smoke"`
`--workers=`	parallel processes	`npx playwright test --workers=4`
`--shard=`	split across CI machines	`npx playwright test --shard=1/3`
`--config=`	use another config (suite file)	`npx playwright test --config=playwright.smoke.config.ts`
`--timeout=`	per-test budget	`npx playwright test --timeout=60000`
`--update-snapshots`	refresh visual baselines	`npx playwright test --update-snapshots`

SECTION 14 — POWER COMBOS (the daily recipes — memorize these five)

```
// 1. Two-part pattern: listen first, then trigger
const [popup] = await Promise.all([
  page.waitForEvent('popup'),
  page.getByRole('link', { name: 'Terms' }).click(),
]);

// 2. Table row: find by content, act on its button
await page.getByRole('row', { name: 'Laptop' })
         .getByRole('button', { name: 'Delete' }).click();

// 3. Wait for the API the UI depends on, then assert
await page.waitForResponse(r => r.url().includes('/api/search') && r.status() === 200);
await expect(page.getByRole('listitem').first()).toBeVisible();

// 4. Mock a failure and assert the UI's error handling
await page.route('**/api/orders', route =>
  route.fulfill({ status: 500, body: '{"error":"down"}' }));
await expect(page.getByRole('alert')).toContainText('Something went wrong');

// 5. Unique data per run (parallel-safe)
const email = `qa_${crypto.randomUUID().slice(0, 8)}@testmail.dev`;
```

FINAL MEMORY HOOK (the whole sheet in one line): *"FIND with getBys, REFINE with filter/nth, DO with click/fill/check/select, READ with innerText/inputValue, VERIFY with expect, WAIT only for URLs and APIs, MOCK with route, LISTEN with Promise.all."*